

Laborator 1 & 2: Introducere



Cuprins

1	<i>Obiective și concepte fundamentale</i>	2
2	<i>Etapale realizării unei aplicații software</i>	2
3	<i>Programarea secvențială și procedurală</i>	3
4	<i>Scheme logice</i>	4
5	<i>Decizie și structuri repetitive</i>	5
6	<i>Pseudocod</i>	6
7	<i>Structura unui program C++</i>	7
8	<i>Citirea, afișarea și preprocesarea</i>	8
9	<i>Spații de nume și exemplu complet</i>	9
10	<i>Aplicații</i>	10

1. Obiective și concepte fundamentale

Obiectivele laboratorului

La finalul laboratorului trebuie să poată fi realizate următoarele operații:

- identificarea datelor de intrare și a datelor de ieșire ale unei probleme;
- formalizarea unei probleme prin variabile, condiții și relații matematice;
- construirea unui algoritm folosind secvențe, decizii și structuri repetitive;
- reprezentarea algoritmului prin schemă logică și pseudocod;
- recunoașterea structurii unui program C++ și a rolului funcției `main()`;
- utilizarea instrucțiunilor, blocurilor, funcțiilor, `cin`, `cout`, `#include` și spațiilor de nume;
- urmărirea relației dintre codul sursă, compilare, depanare și testare.

2. Etapele realizării unei aplicații software

Realizarea unui program poate fi privită ca o succesiune de etape, fiecare având un rezultat intermediar clar:

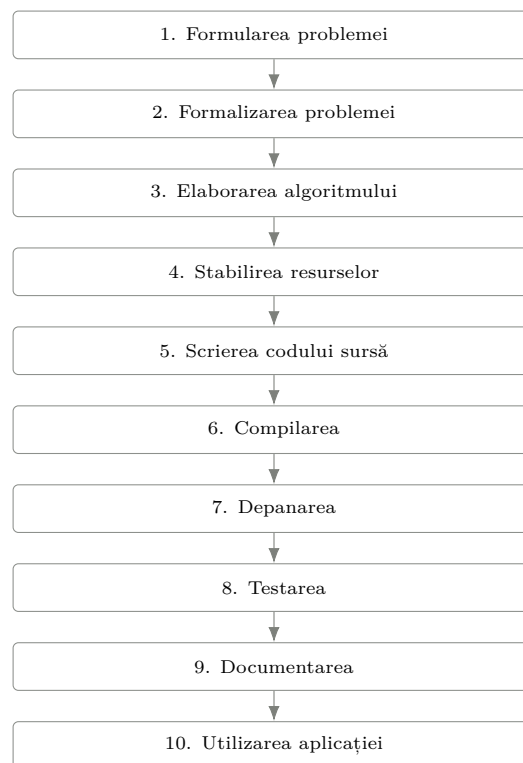


Figura 1: Succesiunea etapelor de realizare a unei aplicații.

2.1. Formularea problemei

Problema trebuie să precizeze ce informații sunt disponibile și ce rezultat se dorește. În forma minimă, sunt identificate **datele de intrare**, **datele de ieșire** și operațiile care transformă intrarea în ieșire. Dacă datele au o structură impusă, aceasta trebuie precizată.

2.2. Formalizarea problemei

Formalizarea transformă descrierea în termeni matematici și algoritmi. Se identifică variabilele și parametrii, tipurile lor, condițiile inițiale, restricțiile și formulele necesare. Pot fi analizate și memoria necesară și timpul de prelucrare.

Exemplu - media a două numere

Pentru două valori reale a și b :

- intrare: a, b ;
- ieșire: m ;
- prelucrare: $m \leftarrow \frac{a+b}{2}$.

Problema formulată în limbaj natural devine o operație matematică precisă și apoi o succesiune de instrucțiuni.

2.3. Algoritmul

Un algoritm este o succesiune de instrucțiuni care urmărește rezolvarea unei probleme. Pentru probleme simple, pașii pot fi executați secvențial; pentru probleme generale apar și ramificări și repetări.

3. Programarea secvențială și procedurală

3.1. Programarea secvențială

Programarea secvențială descrie pașii care trebuie executați unul după altul pentru a ajunge de la starea inițială la rezultatul dorit. Într-un astfel de algoritm, fiecare pas produce o nouă stare a datelor.



Figura 2: Modelul unei prelucrări strict secvențiale.

3.2. Programarea procedurală

Programarea procedurală este bazată pe utilizarea procedurilor sau funcțiilor. O funcție grupează instrucțiuni care realizează o sarcină și poate fi apelată dintr-o altă zonă a programului. Descompunerea problemei în operații mai mici permite organizarea și reutilizarea codului.

Funcție

O funcție este un bloc de instrucțiuni căruia i se asociază un nume; poate primi parametri și poate returna un rezultat.

Exemplu - calculul maximului printr-o funcție

```

1 int maxim(int a, int b)
2 {
3     if (a > b)
4         return a;
5     return b;
6 }
7
8 int main()
9 {
10    cout << maxim(8, 5);
11    return 0;
12 }
  
```

Funcția `maxim` izolează o operație bine definită: compararea a două valori și întoarcerea celei mai mari.

3.3. De la problemă la algoritm

Un mod practic de lucru este:

1. se notează datele de intrare;
2. se notează rezultatul cerut;

3. se scriu formulele sau regulile de calcul;
4. se stabilesc cazurile particulare;
5. se ordonează pașii;
6. se verifică algoritmul pe exemple concrete;
7. abia apoi se scrie codul.

Exemplu - rezolvarea ecuației $ax + b = 0$

Date de intrare: a, b . Rezultatul depinde de valoarea lui a :

- dacă $a \neq 0$, există soluția $x = -\frac{b}{a}$;
- dacă $a = 0$ și $b = 0$, ecuația este nedeterminată;
- dacă $a = 0$ și $b \neq 0$, ecuația este imposibilă.

Problema introduce natural structura de decizie.

4. Scheme logice

O schemă logică reprezintă algoritmul folosind blocuri grafice și săgeți. Fiecare bloc are o semnificație precisă, iar traseul săgeților indică ordinea executării.

4.1. Blocurile de bază

Bloc	Reprezentare	Utilizare
Pornire / oprire	START, STOP în oval	delimitează algoritmul
Citire	paralelogram	preia date de intrare
Scriere	paralelogram	afișează date de ieșire
Atribuire	dreptunghi	calculează și memorează o valoare
Decizie	romb	verifică o condiție

Tabela 1: Blocurile fundamentale ale unei scheme logice.

4.2. Exemplu complet - maximul dintre două numere

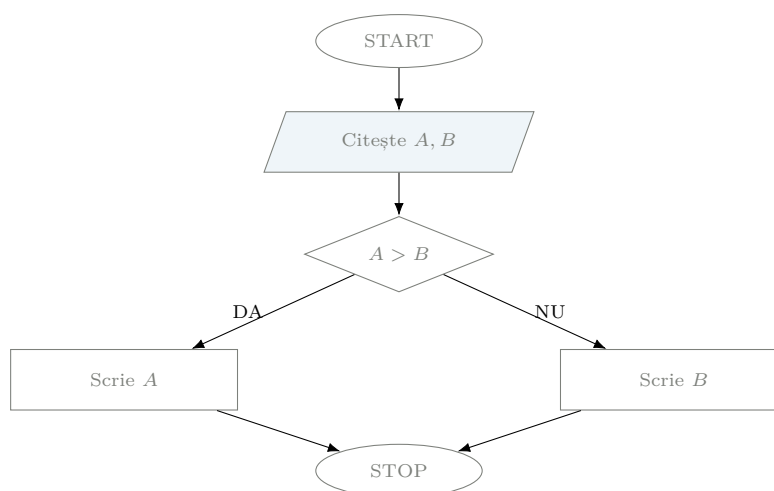


Figura 3: Schema logică pentru calculul maximului dintre A și B .

4.3. Trasarea algoritmului

Pentru $A = 3$, $B = 9$ se parcurg blocurile $\text{START} \rightarrow \text{Citește} \rightarrow A > B \rightarrow \text{ramura NU} \rightarrow \text{Scrie B} \rightarrow \text{STOP}$. Rezultatul este 9.

Regulă practică

Înainte de implementare, fiecare ramură a unei scheme logice trebuie să aibă un traseu clar până la ieșire. Pentru un algoritm cu cicluri trebuie verificat și faptul că există o condiție care permite terminarea buclei.

5. Decizie și structuri repetitive

5.1. Instrucțiunea decizională

Într-o decizie se testează o expresie logică. Dacă aceasta este adevărată se execută o secvență; în caz contrar se execută o altă secvență sau se continuă direct după decizie.

```
1 Daca expresie_logica atunci
2     bloc_instructiuni
3 Altfel
4     bloc_instructiuni
5 sfDaca
```

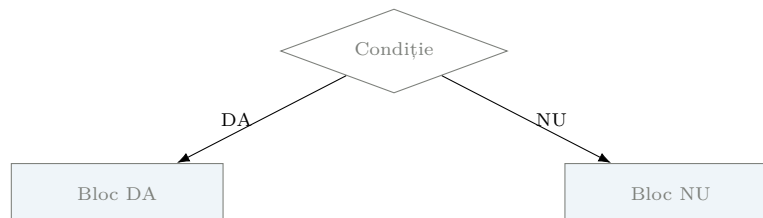


Figura 4: Structura unei decizii.

5.2. Ciclu cu contor

Forma de bază este:

```
1 Pentru contor = V_inicial, V_final, [pas] executa
2     bloc_instructiuni
3 sfPentru
```

Este potrivit când numărul de pași este legat de o variabilă-contor.

Exemplu - afișarea numerelor 1..10

```
1 Pentru i = 1, 10 executa
2     Scrie i
3 sfPentru
```

Valorile lui i sunt 1,2,3,...,10.

5.3. Ciclu cu test inițial

```
1 CatTimp expresie_logica executa
2     bloc_instructiuni
3 sfCatTimp
```

Condiția este verificată înaintea fiecărei execuții a corpului. Dacă este falsă de la început, corpul nu se execută.

5.4. Ciclu cu test final

```
1 Repeta
2     bloc_instructiuni
3 CatTimp expresie_logica
```

Corpul se execută cel puțin o dată, deoarece testul apare după secvența repetată.

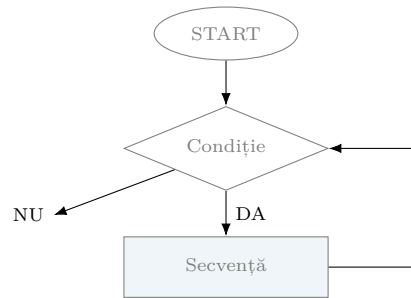


Figura 5: Schema de principiu a unei repetări cu test inițial.

6. Pseudocod

Pseudocodul descrie pașii algoritmului într-o formă apropiată limbajului natural și limbajelor de programare. Sintaxa comenzilor este standardizată, dar nu depinde de un compilator C++.

6.1. Comenzile de bază

Operație	Formă	Exemplu
Citire	Citeste lista de date	Citeste a,b
Scriere	Scrie lista de date	Scrie m
Atribuire	variabila <- expresie	m <- (a+b)/2
Decizie	Daca ... atunci ... Altfel	maximul a două valori
Contor	Pentru ... executa	afișarea 1..10
Test inițial	CatTimp ... executa	sumă până la o condiție
Test final	Repeta ... CatTimp	validarea unei citiri

Tabela 2: Set minimal de comenzi pseudocod.

6.2. Exemplu - minimul dintre două numere

```

1 Citeste a,b
2 min <- a
3 Daca b < min atunci
4     min <- b
5 sfDaca
6 Scrie min
  
```

Algoritmul pornește de la presupunerea că primul număr este minimul curent și actualizează această valoare dacă al doilea număr este mai mic.

6.3. Exemplu - suma primelor n numere

```

1 Citeste n
2 s <- 0
3 Pentru i = 1, n executa
4     s <- s + i
5 sfPentru
6 Scrie s
  
```

Pentru $n = 5$ se obțin succesiv valorile 1, 3, 6, 10, 15, iar rezultatul este 15.

6.4. Pseudocod și schemă logică

Cele două reprezentări descriu aceeași logică în forme diferite. Schema logică este orientată spre fluxul de execuție, în timp ce pseudocodul pune accent pe succesiunea comenzilor. În ambele cazuri trebuie să poată fi urmărite clar intrarea, prelucrarea, condițiile și ieșirea.

Exemplu - factorial

Pentru $n \geq 1$:

```

1 Citeste n
2 p <- 1
3 Pentru i = 1, n executa
4     p <- p * i
5 sfPentru
6 Scrie p

```

Pentru $n = 5$: $p = 1 \cdot 2 \cdot 3 \cdot 4 \cdot 5 = 120$.

7. Structura unui program C++

Programarea înseamnă implementarea unui algoritm folosind sintaxa unui limbaj de programare. Un program C++ este alcătuit din una sau mai multe funcții, iar funcția principală `main()` este obligatorie.

7.1. Structura generală

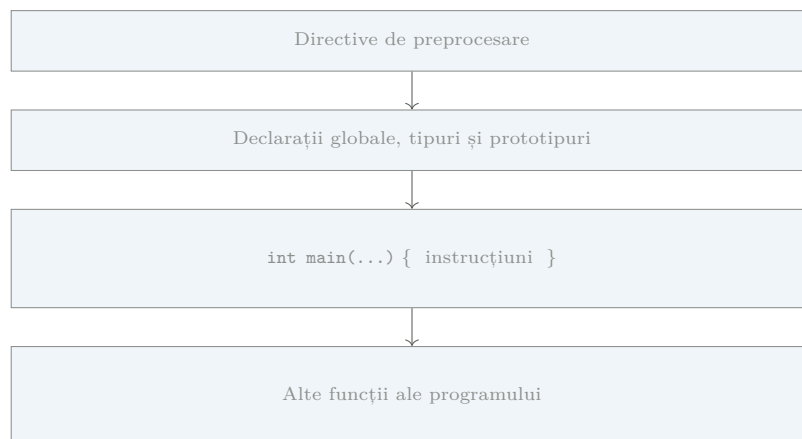


Figura 6: Structura simplificată a unui program C++.

7.2. Funcția main()

Execuția unui program începe în `main()`. Instrucțiunile acesteia sunt parcurse succesiv, în ordinea în care apar.

```

1 #include <iostream>
2
3 int main()
4 {
5     std::cout << "Salut!" << std::endl;
6     return 0;
7 }

```

7.3. Instrucțiuni și blocuri

O instrucțiune specifică o acțiune sau o operație executată de program și se încheie cu `;`. Un bloc este un grup de instrucțiuni delimitat de `{` și `}`. Un bloc poate conține alte blocuri și variabile locale.

```

1 int main()
2 {
3     int a = 5;
4     int b = 8;
5
6     {
7         int suma = a + b;
8         std::cout << suma << std::endl;
9     }
}

```

```

10
11     return 0;
12 }

```

Variabila `suma` este locală blocului interior și nu poate fi utilizată după închiderea aceluiași bloc.

7.4. Funcții

O funcție are forma generală:

```

1 tip_returnat nume_functie(lista_parametri)
2 {
3     // instructiuni
4 }

```

Exemplu: `int produs(int a, int b)` primește două argumente și returnează produsul lor.

8. Citirea, afișarea și preprocesarea

8.1. Afișarea datelor

Obiectul standard `cout` este folosit pentru afișare, iar operatorul `<<` transferă datele către fluxul de ieșire.

```

1 #include <iostream>
2
3 int main()
4 {
5     int a = 7;
6     std::cout << "a = " << a << std::endl;
7     return 0;
8 }

```

8.2. Citirea datelor

Obiectul standard `cin` este folosit pentru citire, iar operatorul `>>` transferă datele în variabila din dreapta.

```

1 #include <iostream>
2
3 int main()
4 {
5     int a, b;
6     std::cin >> a >> b;
7     std::cout << a + b << std::endl;
8     return 0;
9 }

```

Exemplu - media a două numere în C++

```

1 #include <iostream>
2
3 int main()
4 {
5     double a, b;
6     std::cin >> a >> b;
7
8     double m = (a + b) / 2;
9     std::cout << m << std::endl;
10
11     return 0;
12 }

```

Acest program implementează direct algoritmul: citire → calcul → afișare.

8.3. Directive de preprocesare

Preprocesarea este o etapă de prelucrare la nivel de text, realizată înaintea compilării. Directivele sunt comenzi care încep cu simbolul `#`.

8.4. Directiva #include

#include indică includerea unui fișier sursă. Forma **<fișier>** caută în directoarele standard, iar forma **"fișier"** caută mai întâi în directorul curent și apoi în directoarele standard.

```
1 #include <iostream>
2 #include "Funcții.h"
```

Fișierele header conțin declarații și alte elemente necesare utilizării unei biblioteci. Bibliotecile permit reutilizarea funcțiilor deja implementate.

8.5. Procesarea codului

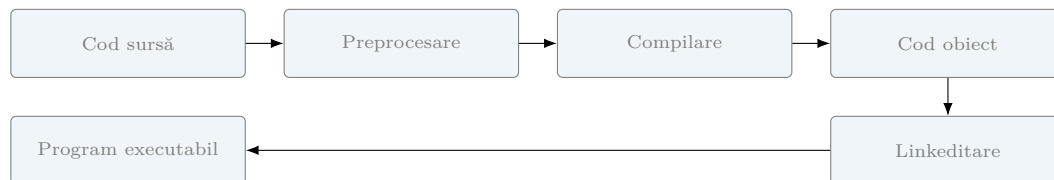


Figura 7: Lanț simplificat de transformare a codului sursă în program executabil.

9. Spații de nume și exemplu complet

9.1. Spațiul de nume

Un spațiu de nume grupează identificatori și permite evitarea conflictelor între elemente cu același nume. Accesul explicit se face cu operatorul **::**.

```
1 namespace Matematica
2 {
3     int dublu(int x)
4     {
5         return 2 * x;
6     }
7 }
8
9 int main()
10 {
11     std::cout << Matematica::dublu(7);
12     return 0;
13 }
```

Forma **using namespace Matematica;** permite utilizarea numelor fără prefix, iar forma **using Matematica::dublu;** introduce un singur identificator în zona curentă.

Exemplu standard

În C++, bibliotecile standard folosesc spațiul de nume **std**. De aceea pot fi întâlnite atât formele **std::cout** și **std::cin**, cât și, în programe introductive, **using namespace std;** urmat de **cout** și **cin**.

9.2. Exemplu complet - numărul prim

Se dorește verificarea dacă un număr natural n este prim.

Algoritm:

```
1 Citeste n
2 prim <- Adevarat
3 Daca n < 2 atunci
4     prim <- Fals
5 Altfel
6     Pentru d = 2, n-1 executa
7         Daca n % d = 0 atunci
8             prim <- Fals
9     sfDaca
10    sfPentru
11 sfDaca
12 Scrie prim
```

O implementare directă în C++ este:

```

1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int n;
7     cin >> n;
8
9     bool prim = true;
10    if (n < 2)
11        prim = false;
12    else
13        for (int d = 2; d < n; d++)
14            if (n % d == 0)
15                prim = false;
16
17    cout << (prim ? "DA" : "NU") << endl;
18    return 0;
19 }

```

9.3. Verificarea exemplului

Pentru $n = 7$, niciuna dintre valorile 2, 3, 4, 5, 6 nu divide pe 7, deci rezultatul este DA. Pentru $n = 12$, valoarea $d = 2$ produce restul 0, astfel rezultatul este NU.

Relația dintre etape

Problemă → identificarea lui n și a criteriului de primalitate → pseudocod → cod C++ → compilare → testare pe valori precum 1, 2, 7, 12, 25.

10. Aplicații

1. Realizați un algoritm în pseudocod și schemă logică pentru afișarea pe ecran a numerelor între 1 și 10.
2. Să se scrie un algoritm pentru determinarea produsului factorial al unui număr n pozitiv (pseudocod și schemă logică).
3. Să se scrie un algoritm recursiv pentru determinarea produsului factorial al unui număr n pozitiv (pseudocod).
4. Realizați un algoritm care calculează media aritmetică a n numere citite de la tastatură (pseudocod și schemă logică).
5. Să se scrie un algoritm care să verifice dacă un număr n natural este prim (pseudocod).
6. Scrieți un algoritm pentru rezolvarea unei ecuații de gradul 2 (schemă logică și pseudocod).
7. Determinați printr-un algoritm cel mai mic număr dintr-un șir de numere citit de la tastatură (schemă logică și pseudocod).
8. Scrieți un algoritm care calculează primele n numere ale șirului Fibonacci (pseudocod).
9. Scrieți un algoritm care calculează recursiv primele n numere ale șirului Fibonacci (pseudocod).

Verificarea soluției

Pentru fiecare problemă se recomandă aceeași ordine de lucru: identificarea intrării și ieșirii, formalizarea, algoritmul, verificarea pe exemple, implementarea în C++, compilarea, depanarea și testarea.

Noțiunile de bază urmărite: algoritm, schemă logică, pseudocod, secvență, decizie, repetare, funcție, main(), intrare/ieșire, preprocesare, biblioteci și spații de nume.